

Software Requirements Specification (SRS)

Project: Online Quiz System — A console-based quiz game with multiple-choice questions and scoring

Version: 1.0

Authors: Chiranth R. (PES1UG24AM354), Tanvi B. Shetty (PES1UG24AM302), Abdul Mateen Shaikh (PES1UG24AM332), Ashvin Ronald Lobo (PES1UG24AM345)

Date: 06-09-2026

Status: Draft

Team members and roles

Role	Name	Responsibilities
Team Lead / Project Coordinator	Chiranth R. (PES1UG24AM354)	Overall coordination, task allocation, integration, and submission management.
Developer — Quiz Engine & Scoring	Abdul Mateen Shaikh (PES1UG24AM332)	Implements quiz gameplay flow, question presentation, answer validation, and scoring logic.
Developer — Question Bank & Admin Module	Tanvi B. Shetty (PES1UG24AM302)	Implements question bank storage/file I/O, admin CRUD operations, and admin authentication.
QA & Documentation Lead	Ashvin Ronald Lobo (PES1UG24AM345)	Test case design/execution, acceptance criteria verification, and SRS/RTM maintenance.

Revision history

Version	Date	Author	Change summary	Approval
1.0	06-09-2026	Chiranth R, Tanvi B. Shetty, Abdul Mateen Shaikh, Ashvin Ronald Lobo	Initial SRS draft for Online Quiz System	

Approvals

Role	Name	Signature / Email	Date
Course Coordinator			

Table of Contents

1. Introduction
2. Overall description
3. External interfaces
4. System features (detailed)
5. Non-functional requirements (detailed)
6. Quality attributes & Acceptance tests
7. System models and diagrams (UML Use-Case Diagram)
8. Requirements Traceability Matrix (RTM)
9. Project Backlog (Epics & User Stories)

1. Introduction

1.1 Purpose

This document is a Software Requirements Specification (SRS) for the Online Quiz System, a console-based (command-line) application that presents multiple-choice questions to a user, accepts their answers, and computes a score. It defines the functional and non-functional requirements, interfaces, and verification criteria for the development team and course evaluators.

1.2 Scope

Covers user interaction at the console for taking quizzes, selection of quiz category/difficulty, automatic scoring and results feedback, storage and retrieval of a local question bank, and an administrator workflow for managing questions. Excludes any graphical user interface, web/mobile interface, networked or multiplayer quiz play, and integration with external learning management systems (LMS) — these are out of scope for this version.

1.3 Audience

Developers, QA/Testers, Course Instructor/Evaluators, and end users (players/students) taking the quiz.

1.4 Definitions

List of acronyms: MCQ (Multiple Choice Question), CLI (Command Line Interface), SRS, ID, I/O, NFR, FR, RTM, UML.

2. Overall description

2.1 Product perspective

The Online Quiz System is a standalone console application with no external network dependency. It reads a local question bank (file-based) and writes quiz results/history to a local file. It does not depend on a database server or external services.

2.2 Major product functions (detailed)

- Display a main menu (Start Quiz, View Leaderboard/History, Admin: Manage Questions, Exit)
- Allow selection of a quiz category and difficulty level
- Present multiple-choice questions one at a time with four answer options
- Accept and validate the user's answer choice
- Calculate and display the score and a results summary at the end of the quiz
- Persist each completed attempt (player, category, score, date/time) to a results file
- Display a leaderboard and a player's own past attempt history
- Provide an admin workflow to add, edit, delete, and view questions in the question bank
- Randomize question order and shuffle answer options per attempt
- Handle invalid input gracefully without crashing

2.3 User roles and characteristics

- Player / Quiz Taker: general user attempting quizzes; no technical skill assumed.
- Administrator: manages the question bank; typically a team member or instructor.
- Developer / Tester: builds and verifies system behaviour.

2.4 Operating environment

Runs as a compiled console executable on Windows, Linux, or macOS terminals. Built with C++17 (or later) and the standard library. Uses the local file system to store the question bank and results/history files. No internet connection required.

2.5 Constraints

Must run without a GUI or internet access; single user per session (no concurrent multi-user access); question bank size limited by local storage and memory; console I/O is text/ASCII based.

3. External interface requirements

3.1 User interfaces

A text-based, menu-driven command-line interface. Menus use numbered or lettered options; prompts and error messages are clear and consistent. Questions and the final score summary are formatted consistently for readability.

3.2 Hardware interfaces

Standard keyboard for input and a monitor/terminal for text output. No special hardware peripherals are required.

3.3 Software interfaces

- Local file system interface to read the question bank file (e.g., questions.csv / .txt / .json).
- Local file interface to persist quiz results and history (e.g., results.csv).
- Standard C++ libraries (iostream, fstream, sstream, vector, algorithm) for I/O, parsing, and randomization.

3.4 Communications

No network communication is required for this version — the application is fully offline. A future extension could export results to a shared file or cloud service, but this is out of scope for v1.0.

4. System features (detailed)

Each requirement below includes acceptance criteria and a reference test case. IDs follow QZ-F-####.

4.1 Quiz Setup & Category Selection

Description: Let the player choose what kind of quiz to take before questions are presented.

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
QZ-F-001	The system shall display a main menu with options to start a quiz, view score history/leaderboard, access the admin panel, and exit.	Functional	High	Course Team	AC-QZ-F-001: All four options are shown and each navigates correctly. Test: TC-Menu-01	
QZ-F-002	The system shall allow the player to select a quiz category (e.g., General Knowledge, Science, Math) before starting.	Functional	High	Player	AC-QZ-F-002: Selected category's questions only are used. Test: TC-Menu-02	Requires categorized question bank
QZ-F-003	The system shall allow the player to select a difficulty	Functional	Medium	Player	AC-QZ-F-003: Only questions matching chosen	

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
	level (Easy/Medium/Hard) before starting a quiz.				difficulty appear. Test: TC-Menu-03	
QZ-F-004	The system shall randomly select a configurable number of questions (default 10) from the chosen category/difficulty without repetition within a session.	Functional	High	Player	AC-QZ-F-004: No duplicate question appears in one attempt. Test: TC-Menu-04	Requires shuffling logic

4.2 Quiz Gameplay

Description: Present questions and capture answers during an active quiz attempt.

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
QZ-F-010	The system shall display one multiple-choice question at a time with exactly four answer options labeled A-D.	Functional	High	Player	AC-QZ-F-010: Question text and 4 labeled options render correctly. Test: TC-Quiz-01	
QZ-F-011	The system shall shuffle the order of answer options for each question on every attempt.	Functional	Medium	QA	AC-QZ-F-011: Option order differs across repeated attempts. Test: TC-Quiz-02	
QZ-F-012	The system shall accept the player's answer as a single character (A-D) and re-prompt on invalid input without penalizing the score.	Functional	High	Player	AC-QZ-F-012: Invalid entry re-prompts; score unaffected. Test: TC-Quiz-03	
QZ-F-013	The system shall enforce an optional, configurable per-question time limit and mark an unanswered question incorrect if the limit is exceeded.	Functional	Low	Course Team	AC-QZ-F-013: Question times out and is marked incorrect. Test: TC-Quiz-04	Optional feature flag
QZ-F-014	The system shall allow the player to skip a question, marking it as unattempted in the results summary.	Functional	Medium	Player	AC-QZ-F-014: Skipped question counted separately from incorrect. Test: TC-Quiz-05	
QZ-F-015	The system shall display immediate feedback (Correct/Incorrect) after each answer when feedback mode is enabled.	Functional	Low	Player	AC-QZ-F-015: Feedback shown immediately after submission. Test: TC-Quiz-06	Configurable setting

4.3 Scoring & Results

Description: Compute, present, and store the outcome of a quiz attempt.

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
QZ-F-020	The system shall calculate the total score as the count	Functional	High	Player	AC-QZ-F-020: Score equals number of correct answers. Test: TC-Score-01	

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
	of correct answers at the end of the quiz.					
QZ-F-021	The system shall display a results summary showing total questions, correct, incorrect, skipped, and percentage score.	Functional	High	Player	AC-QZ-F-021: Summary values reconcile with total questions. Test: TC-Score-02	
QZ-F-022	The system shall assign a grade/remark (e.g., Excellent/Good/Needs Improvement) based on the percentage score.	Functional	Medium	Course Team	AC-QZ-F-022: Remark matches the defined score bands. Test: TC-Score-03	
QZ-F-023	The system shall save each completed quiz attempt (player name, category, score, date/time) to a persistent results file.	Functional	High	Course Team	AC-QZ-F-023: New record appended to results file after each attempt. Test: TC-Score-04	Depends on file I/O module
QZ-F-024	The system shall display a leaderboard of the top N scores (all-time or per category) on request.	Functional	Medium	Player	AC-QZ-F-024: Leaderboard sorted descending by score. Test: TC-Score-05	
QZ-F-025	The system shall allow a player to view their own past attempt history on request.	Functional	Medium	Player	AC-QZ-F-025: History filtered to the requesting player's name. Test: TC-Score-06	

4.4 Question Bank Administration

Description: Allow an administrator to maintain the question bank used by the quiz engine.

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
QZ-F-030	The system shall provide an admin menu protected by a password to manage the question bank.	Functional	High	Admin	AC-QZ-F-030: Admin menu inaccessible without correct password. Test: TC-Admin-01	See QZ-SR-001
QZ-F-031	The system shall allow the admin to add a new question with four options, the correct answer, category, and difficulty.	Functional	High	Admin	AC-QZ-F-031: New question appears in the bank and is usable in quizzes. Test: TC-Admin-02	
QZ-F-032	The system shall allow the admin to edit or delete an existing question by its ID.	Functional	High	Admin	AC-QZ-F-032: Edited/deleted question reflected on next load. Test: TC-Admin-03	
QZ-F-033	The system shall allow the admin to view all questions in the bank, optionally filtered by category or difficulty.	Functional	Medium	Admin	AC-QZ-F-033: Filtered list matches selected criteria. Test: TC-Admin-04	
QZ-F-034	The system shall validate that a new/edited question has exactly one correct answer marked and non-	Functional	High	QA	AC-QZ-F-034: Invalid question rejected with a clear error. Test: TC-Admin-05	

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
	empty option text before saving.					

4.5 Data Persistence & Error Handling

Description: Ensure data is loaded/saved reliably and the application degrades gracefully on bad input or file errors.

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
QZ-F-040	The system shall load the question bank from a local file at startup and report a clear error if the file is missing or corrupted.	Functional	High	Course Team	AC-QZ-F-040: Missing/corrupt file produces a readable error, not a crash. Test: TC-Data-01	
QZ-F-041	The system shall persist all question bank changes to the file immediately after an admin edit.	Functional	High	Admin	AC-QZ-F-041: File on disk reflects change without requiring app restart. Test: TC-Data-02	
QZ-F-042	The system shall handle unexpected input (non-numeric or out-of-range menu choices) without crashing, re-prompting the user instead.	Functional	High	QA	AC-QZ-F-042: App remains responsive after invalid input. Test: TC-Data-03	

Player Quiz Execution Flowchart

5. Non-functional requirements (detailed)

NFRs below are measurable and tied to test plans. IDs QZ-NF-####.

Req ID	Requirement	Category	Priority	Acceptance criteria / Measurement
QZ-NF-001	The system shall load the question bank and display the first question within 2 seconds on a standard machine.	Performance	High	Startup-to-first-question time $\leq$ 2s in test runs. Test: TC-Perf-01
QZ-NF-002	The system shall not lose previously saved quiz results or question data if the application terminates unexpectedly mid-session.	Reliability	High	Forced-termination test shows prior saves intact. Test: TC-Rel-01
QZ-NF-003	The system shall present clear, unambiguous prompts and error messages so a first-time user can complete a quiz without external instructions.	Usability	Medium	Usability walkthrough with a new user completed without help. Test: TC-UX-01

Req ID	Requirement	Category	Priority	Acceptance criteria / Measurement
QZ-NF-004	The system's source code shall be modularized (separate modules/classes for menu, quiz engine, scoring, and file I/O) so the 4-person team can work independently.	Maintainability	Medium	Code review confirms module separation and low coupling. Test: TC-Maint-01
QZ-NF-005	The system shall compile and run on Windows, Linux, and macOS using a standard C++17 compiler without platform-specific dependencies.	Portability	Medium	Successful build/run verified on all three OS targets. Test: TC-Port-01
QZ-NF-006	The question bank file format shall support adding new categories or questions without requiring code changes.	Scalability/Maintainability	Low	New category added via data file only; app picks it up on next run. Test: TC-Maint-02

5.1 Security

5.1.1 Security Objectives

- SO-1: Protect the question bank and admin functions from unauthorized modification by restricting access via an admin password.
- SO-2: Preserve the integrity of stored quiz results so past scores cannot be silently altered through the normal application interface.
- SO-3: Preserve quiz fairness by preventing a player from viewing correct answers or the raw question bank contents through the quiz-taking interface.

5.1.2 Security Requirements

Req ID	Requirement (shall...)	Type	Priority	Acceptance criteria / Test case ref
QZ-SR-001	The system shall require a valid admin password before allowing access to add, edit, or delete questions.	Security	High	AC-QZ-SR-001: Admin actions blocked without correct password. Test: TC-Sec-01
QZ-SR-002	The system shall not reveal the correct answer to a question until after the player has submitted or skipped their response.	Security	High	AC-QZ-SR-002: Correct answer absent from on-screen output before submission. Test: TC-Sec-02
QZ-SR-003	The system shall store the admin password in a non-plaintext form (e.g., hashed) rather than as plain readable text in source or config files.	Security	Medium	AC-QZ-SR-003: Password value not recoverable by inspecting stored files. Test: TC-Sec-03
QZ-SR-004	The system shall validate and sanitize all file-based inputs (question bank, results file) and reject malformed records instead of crashing or corrupting data.	Security	High	AC-QZ-SR-004: Malformed record logged and skipped; app continues running. Test: TC-Sec-04
QZ-SR-005	The system shall restrict question-bank write access to the admin workflow only; no path from the quiz-taking menu shall allow editing scores or questions.	Security	High	AC-QZ-SR-005: Code/UI review confirms no player-facing edit path exists. Test: TC-Sec-05
QZ-SR-006	The system shall lock out the admin menu after 3 consecutive incorrect password attempts for the remainder of the session.	Security	Medium	AC-QZ-SR-006: 4th consecutive attempt is blocked until restart. Test: TC-Sec-06

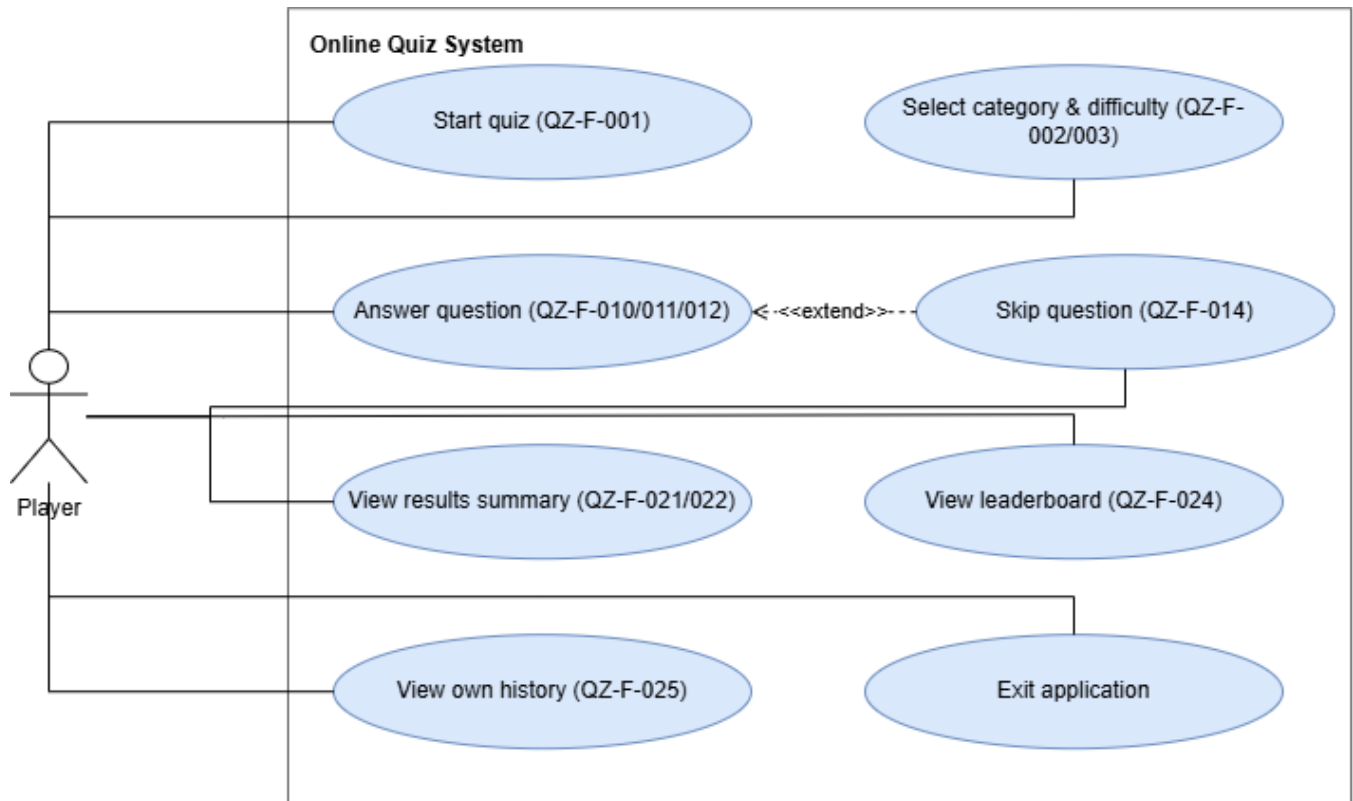
6. Quality attributes & Acceptance tests

Exit criteria for acceptance: all high-priority functional requirements implemented and verified, no critical NFR failures, and the RTM shows all test cases passed.

Acceptance test suites: Menu Navigation, Quiz Gameplay, Scoring & Results, Admin Question Management, Data Persistence, Security (admin authentication), and Usability.

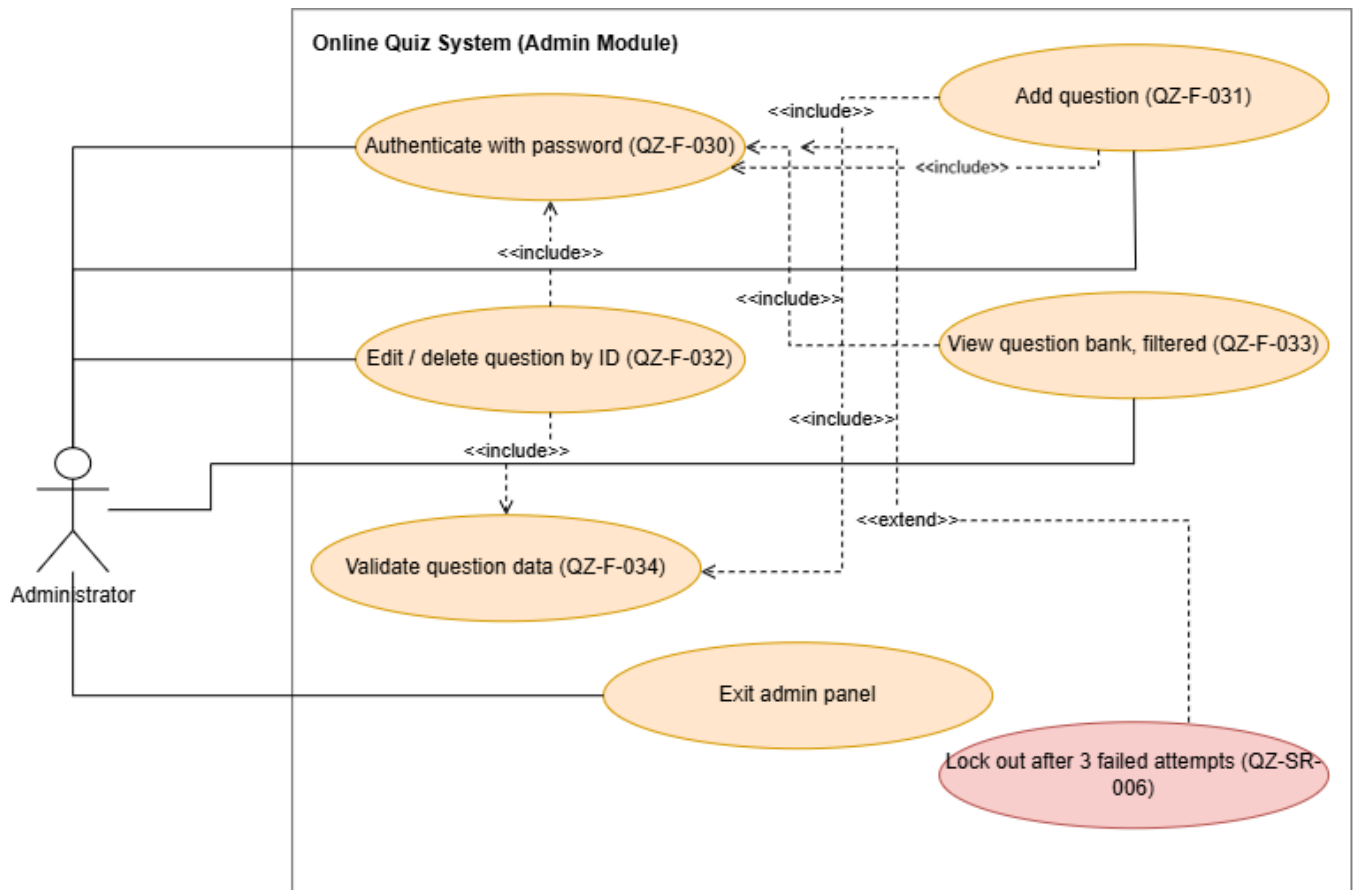
7. System models and diagrams

7.1 Player Use-Case Diagram



The diagram illustrates the interactions between the Player and the Online Quiz System. The player can start a quiz, select category and difficulty, answer or skip questions, view results, access the leaderboard, view quiz history and exit the application.

7.2 Administrator Use-Case Diagram



The diagram illustrates the interactions between the Administrator and the Online Quiz System. The administrator must authenticate before accessing question-management operations and can add, edit, delete and view questions subject to validation and security controls.

8. Requirements Traceability Matrix (RTM)

Req ID	Requirement short	Section ref / Design Spec	Module	Test case(s)	Status (N/P/A)	Comments
QZ-F-001	Main menu display	4.1 / DS-Menu-01	MenuModule	TC-Menu-01	N	
QZ-F-004	Random question selection	4.1 / DS-Menu-04	QuizEngine	TC-Menu-04	N	
QZ-F-010	Display MCQ	4.2 / DS-Quiz-01	QuizEngine	TC-Quiz-01	N	
QZ-F-020	Calculate score	4.3 / DS-Score-01	ScoringModule	TC-Score-01	N	
QZ-F-023	Persist attempt	4.3 / DS-Score-04	FileIOModule	TC-Score-04	N	
QZ-F-030	Admin authentication	4.4 / DS-Admin-01	AdminModule	TC-Admin-01	N	See QZ-SR-001
QZ-F-040	Load question bank	4.5 / DS-Data-01	FileIOModule	TC-Data-01	N	

Req ID	Requirement short	Section ref / Design Spec	Module	Test case(s)	Status (N/P/A)	Comments
QZ-NF-001	Startup response time	5 / DS-Perf-01	FileIOModule / QuizEngine	TC-Perf-01	N	
QZ-NF-005	Cross-platform build	5 / DS-Port-01	Build System	TC-Port-01	N	
QZ-SR-001	Admin password required	5.1.2 / DS-Sec-01	AdminModule	TC-Sec-01	N	
QZ-SR-002	Hide correct answer pre-submit	5.1.2 / DS-Sec-02	QuizEngine	TC-Sec-02	N	

9. Project Backlog (Epics & User Stories)

The epics and user stories below are tracked in Jira (project OQSSMP, board 68) and reflect the team's current sprint backlog. All items are shown as of 06-09-2026; status and priority will change as work progresses — refer to Jira for the live board.

9.1 Epic OQSSMP-1: Core Quiz Engine

Handles the main game loop - presenting questions one at a time, capturing user input, validating answers, and controlling overall game flow from start to finish.

Key	Summary	Status	Priority
OQSSMP-11	Implement main game loop / flow control	To Do	Medium
OQSSMP-12	Implement answer validation logic	To Do	Medium

9.2 Epic OQSSMP-2: Question Bank & Persistence

Manages the question data itself - schema (question, options, correct answer, category/difficulty), storage format (file/JSON/DB), and loading/saving logic for the question set.

Key	Summary	Status	Priority
OQSSMP-13	Define question schema (question, options, answer, category, difficulty)	To Do	Medium
OQSSMP-14	Implement load questions from file/JSON	To Do	Medium
OQSSMP-15	Populate sample question set (20-30 Qs)	To Do	Medium

9.3 Epic OQSSMP-3: Console UI/Input Handling

Covers everything the player sees and types - formatting questions/options on screen, validating and sanitizing input, and displaying end-of-game summaries.

Key	Summary	Status	Priority
OQSSMP-16	Display question and options on screen	To Do	Medium
OQSSMP-17	Implement input validation (reject invalid choices)	To Do	Medium
OQSSMP-18	Build end-of-game summary screen	To Do	Medium

9.4 Epic OQSSMP-4: Scoring & Results

Calculates and tracks the player's score (including any bonus rules like streaks or speed), and handles storing/displaying results (e.g. high-score persistence, leaderboard).

Key	Summary	Status	Priority
OQSSMP-19	Implement score calculation logic (incl. bonus rules if any)	To Do	Medium
OQSSMP-20	Implement high-score/leaderboard persistence	To Do	Medium

9.5 Epic OQSSMP-5: Security Requirements

Addresses protection of the question bank and game data - preventing tampering with the answer key file, input sanitization against malformed/malicious input, safe file handling.

Key	Summary	Status	Priority
OQSSMP-21	Implement input sanitization	To Do	Medium
OQSSMP-22	Prevent tampering with answer key/question file	To Do	Medium

9.6 Epic OQSSMP-6: Non-Functional Requirements

Covers quality attributes outside specific features - performance (response time per question), reliability (no crashes on bad input), usability, and maintainability of the codebase.

Key	Summary	Status	Priority
OQSSMP-23	Define and verify response-time requirement per question	To Do	Medium
OQSSMP-24	Implement reliability/error handling (no crash on bad input)	To Do	Medium

9.7 Epic OQSSMP-7: Testing & QA / RTM

Owns writing and maintaining test cases for each requirement, tracking pass/fail status, and keeping the Requirements Traceability Matrix (RTM) updated as work progresses.

Key	Summary	Status	Priority
OQSSMP-25	Write unit tests for scoring logic	To Do	Medium
OQSSMP-26	Write unit tests for input/answer validation	To Do	Medium
OQSSMP-27	Set up and maintain RTM tracking sheet	To Do	Medium

9.8 Epic OQSSMP-8: Documentation (SRS + UML)

Covers producing the SRS document itself and the accompanying UML diagrams (use-case diagrams, etc.) that get inserted into it.

Key	Summary	Status	Priority
OQSSMP-28	Draft SRS content (all sections)	To Do	Medium
OQSSMP-29	Create UML use-case diagrams	To Do	Medium
OQSSMP-30	Compile final SRS docx with diagrams inserted	To Do	Medium